

23CSE312-DS-Lab Activity

Distributed Systems Lab

Group Formation Instructions

1. Students must form groups of **minimum 3 and maximum 5 members**.
2. **Group formation is mandatory.**
Individual submissions will **not be accepted**.
3. Each group must:
 - Work on **all five modules**
 - Ensure **equal contribution** from all members
(Every member should be able to explain any module during evaluation.)
4. **One group leader** must be identified:
 - Responsible for coordination
 - Responsible for final submission
 - Point of contact during lab evaluation
5. **Group details must be submitted before the end of this lab session:**
 - Names and roll numbers of all members
6. **No group changes are allowed after submission**
(except with prior instructor approval).
7. During the **live demo**, any member of the group may be asked to:
 - Explain system architecture
 - Run the application
 - Answer questions on distributed system concepts
8. In cases of plagiarism or unequal contribution:
 - Marks may be reduced
 - Individual members may be evaluated separately



Lab Activity: Hostel Assist – Distributed Web Application

Objective

Design and implement a **realistic, usable hostel utility application** using **Distributed Systems concepts**.

Students will work in **groups (maximum 5 members)**.

Each group must implement **all modules**, where **each module demonstrates a different distributed communication model**, along with a **basic user interface** and **in-memory data management**.

This lab emphasizes:

- Distributed communication models
 - System design and decomposition
 - State and concurrency management
 - Practical, demo-ready applications
-

General Rules (Applicable to ALL Groups)

1. User Interface is Mandatory

- Web UI (HTML / CSS / JavaScript) **OR**
- Desktop UI (JavaFX / Swing)
- UI aesthetics are **not graded**; functionality is mandatory

2. In-Memory Storage is Mandatory

- Use collections (List, Map, Array, etc.)
- Databases are **not required**
- Database usage (if any) will be treated as an **enhancement only**



3. Application must be:
 - Runnable
 - Demo-ready
 - Stable during evaluation
 4. Focus must be on **Distributed Systems concepts**, not UI polish
 5. Each group must clearly **justify why in-memory storage is appropriate** for each module
-

Important Design Note

In real-world distributed systems, **not all data needs to be persistent**.

Examples:

- Live dashboards
- Session-based services
- Caching layers
- Temporary coordination services

Your design must explain:

- What data is stored in memory
 - Why persistence is unnecessary for that module
 - How system behavior is affected if the server restarts
-

Modules

Module 1: Socket Programming

Module Name: Hostel Complaint Management System

Functionality

- Students submit hostel complaints via UI



- Complaint details include:
 - Room number
 - Category (Water / Electricity / Cleanliness / Other)
 - Description
- Server receives complaints using **socket programming**
- Server sends acknowledgment to the client

UI Requirements

- Complaint submission form
- Complaint confirmation / status display

Technical Requirements

- Client–server socket communication
- Server must handle **multiple clients**
- Complaints stored in an **in-memory data structure** (List / Array)

Learning Focus

- Socket-based communication
 - Concurrent client handling
 - Server-side state management
-

Module 2: Java RMI

Module Name: Hostel Room Information Service

Functionality

- Students search for hostel room details via UI



- Information returned includes:
 - Room number
 - Occupant names
 - Warden contact details
- Data retrieved using **Java RMI**

UI Requirements

- Room number input field
- Display of room and warden details

Technical Requirements

- RMI registry setup
- At least **two remote methods**
- In-memory storage using Map / object collections

Learning Focus

- Remote Method Invocation
 - Distributed object management
 - Stub–skeleton interaction
-

Module 3: RPC (REST API)

Module Name: Hostel Notice Board System

Functionality

- Admin adds hostel notices
- Students view notices via UI



- Notice details include:
 - Title
 - Message
 - Date

UI Requirements

- Admin page to add notices
- Student page to view notices

Technical Requirements

- RESTful API (GET, POST)
- Stateless request–response model
- In-memory list of notices

Learning Focus

- RPC principles
 - Stateless service design
 - Client–server interaction over HTTP
-

Module 4: Peer-to-Peer (P2P)

Module Name: Student Resource Sharing System

Functionality

- Students share academic resources (PDFs, notes)
- Each peer can:
 - Upload a file
 - Download files from another peer
- No centralized file storage server

UI Requirements



- File upload interface
- Peer discovery / selection
- Download functionality

Technical Requirements

- Each peer acts as **both client and server**
- Direct peer-to-peer communication
- File transfer over the network

Learning Focus

- Decentralized architectures
 - Peer discovery mechanisms
 - Fault tolerance and availability
-

Module 5: Shared Memory

Module Name: Mess Feedback Live Counter

Functionality

- Students submit mess feedback:
 - Good
 - Average
 - Poor
- Live feedback counts displayed on UI
- Multiple users update simultaneously

UI Requirements

- Feedback buttons
- Live result display

Technical Requirements



- Shared memory segment
- Proper synchronization (Semaphore / Mutex)
- Multiple processes accessing shared data safely

Learning Focus

- Shared memory concepts
 - Synchronization techniques
 - Race condition prevention
-

Deliverables (MANDATORY)

Each group must submit:

1. **Complete source code**
 2. **README file** containing:
 - Problem description
 - Communication model used for each module
 - In-memory data design explanation
 - Steps to compile and run the application
 3. **Architecture diagram**
 - Clearly showing communication models
 - Client–server / peer interactions
 4. **Live demonstration during lab evaluation**
-

Strictly Not Allowed

- Copying code from shared or reference applications
- Using a full-fledged database as core storage
- Submitting UI without backend logic

